Refactoring is where you think about your code and how best to structure it. Skipping refactoring will slowly start to hurt, and by the time you notice the problem, it'll be much harder to clean up than if you had addressed it early.

Prescription 13	Refactoring is where a lot of design happens in TDD, and it's easiest to do in small steps. Skip it at your peril.

At the most abstract level, you're looking for three things: complexity to break up, duplication to combine, and abstractions waiting to be born.

Break Up Complexity

Complexity will often initially manifest itself as long methods or long lines of code. If you're doing a "quick to green" first pass of the code to make the tests pass, there will often be a line of code with lots of method chaining or the like. It's almost always a good idea to break up long methods or long lines by extracting part of the code into its own method. In addition to simplifying the original method, you have the opportunity to give the method you're creating a name that is meaningful in the domain and that can make your code self-documenting.

Booleans, local variables, and inline comments are almost always candidates for extraction:

- Any compound Boolean logic expression goes in its own method. It's much harder than you might think for people to understand what a compound expression does. Give people reading your code a fighting chance by hiding the expression behind a name that expresses the code's intent, such as `valid_name?` or `has_purchased_before?`.
- Local variables are relatively easy to break out into methods with the same name as the variable—in Ruby, code that uses the variable doesn't need to change if the variable becomes a method with no arguments. Having a lot of local variables is a huge drag on complex refactorings. You'll be surprised at how much more flexible your code feels if you minimize the number of local variables in methods. (I first encountered this idea, along with a lot of other great refactoring ideas, in *Refactoring: Improving the Design of Existing Code [FBBO99]*.)
- In long methods, sometimes a single-line comment breaks up the method by describing what the next part does. This nearly always is better extracted to a separate method with a name based on the comment's contents. Instead of one twenty-five-line method, you wind up with a five-line